Science and Technology Facilities Council

CRYSTAL In Parallel

Ian Bush
STFC
RAL
Ian.Bush@stfc.ac.uk

CoSeC
Computational Science Centre for Research Communities

Introduction

- Why Parallel?
- What is in a Parallel Computer?
- When Parallel?
- Pcrystal
- MPPcrystal
- How to get the best out of parallel Crystal
- Examples of MPPcrystal calculations

Why Parallel?

- Actually two questions
  - Why do we need parallel computers? Can't we build just a big, really powerful “normal” serial one?
  - Why should I be interested in using parallel computers?

Do We Need Parallel

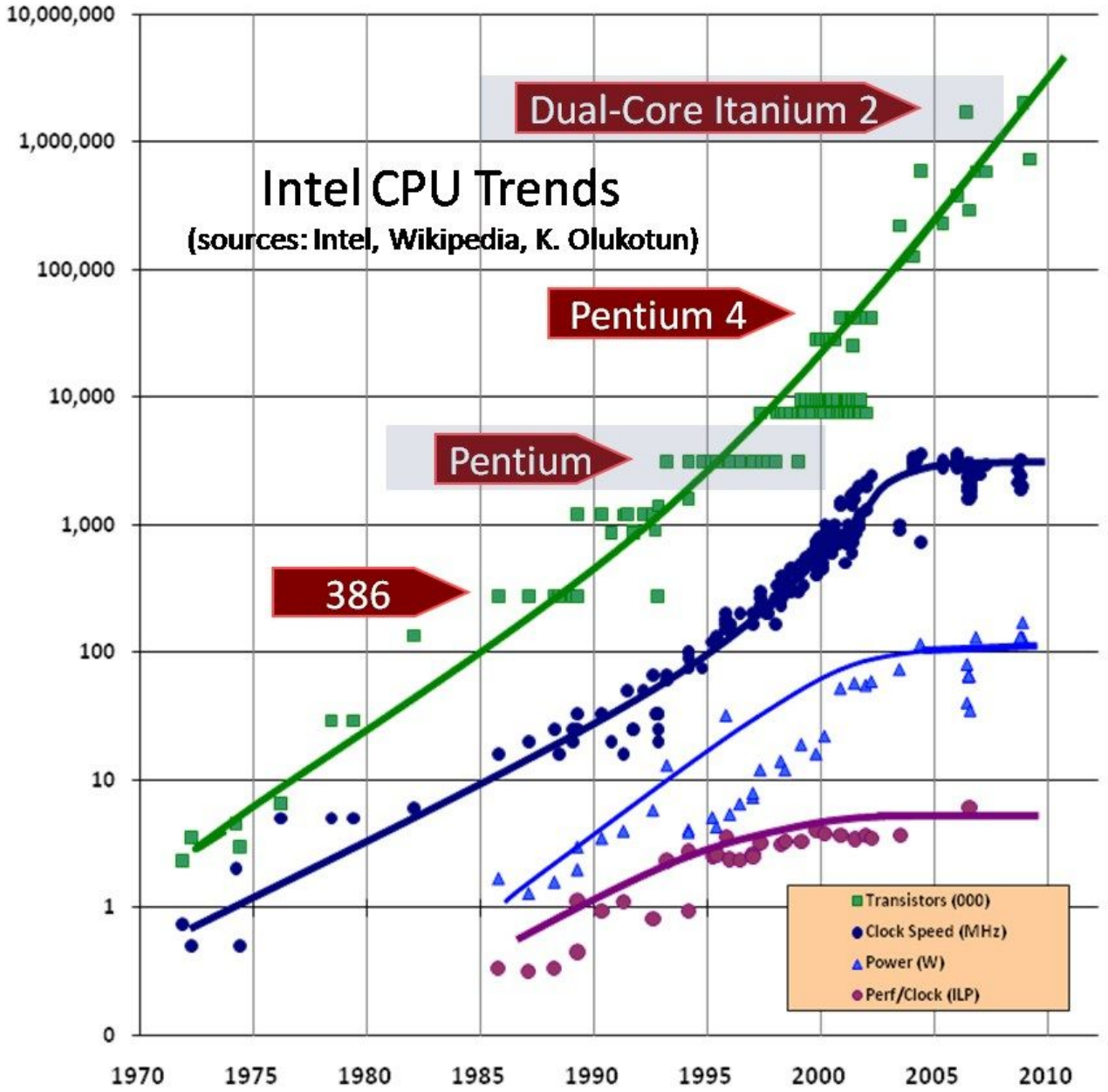
If we didn't go parallel the machine would melt!

- Just can't cool them well enough

All modern computers are parallel nowadays - some examples:

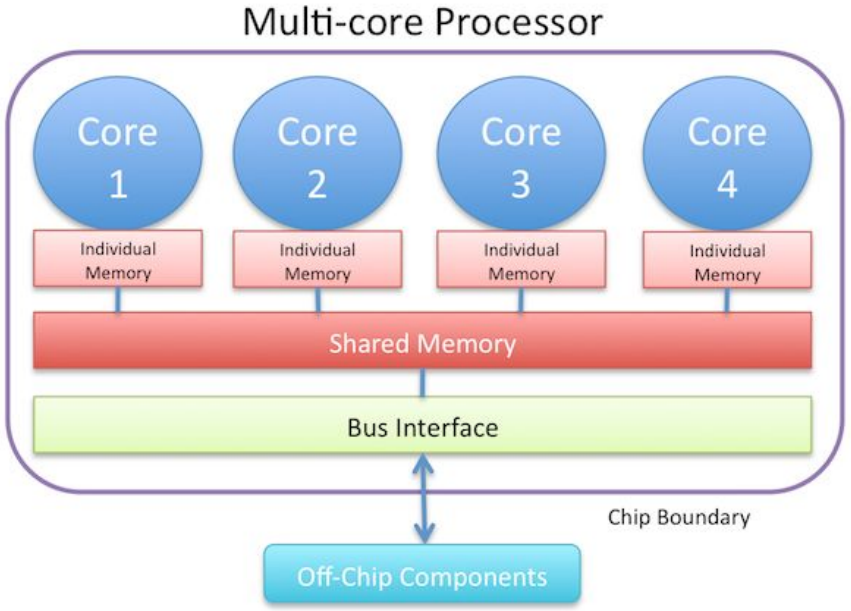
Dualit.

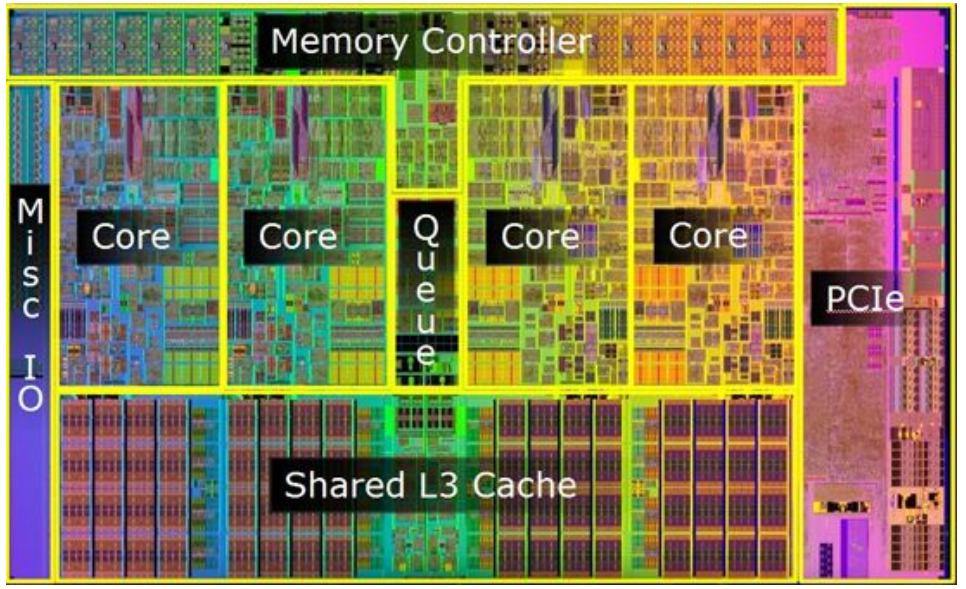
Science and Technology Facilities Council



What Is In A Parallel Computer?

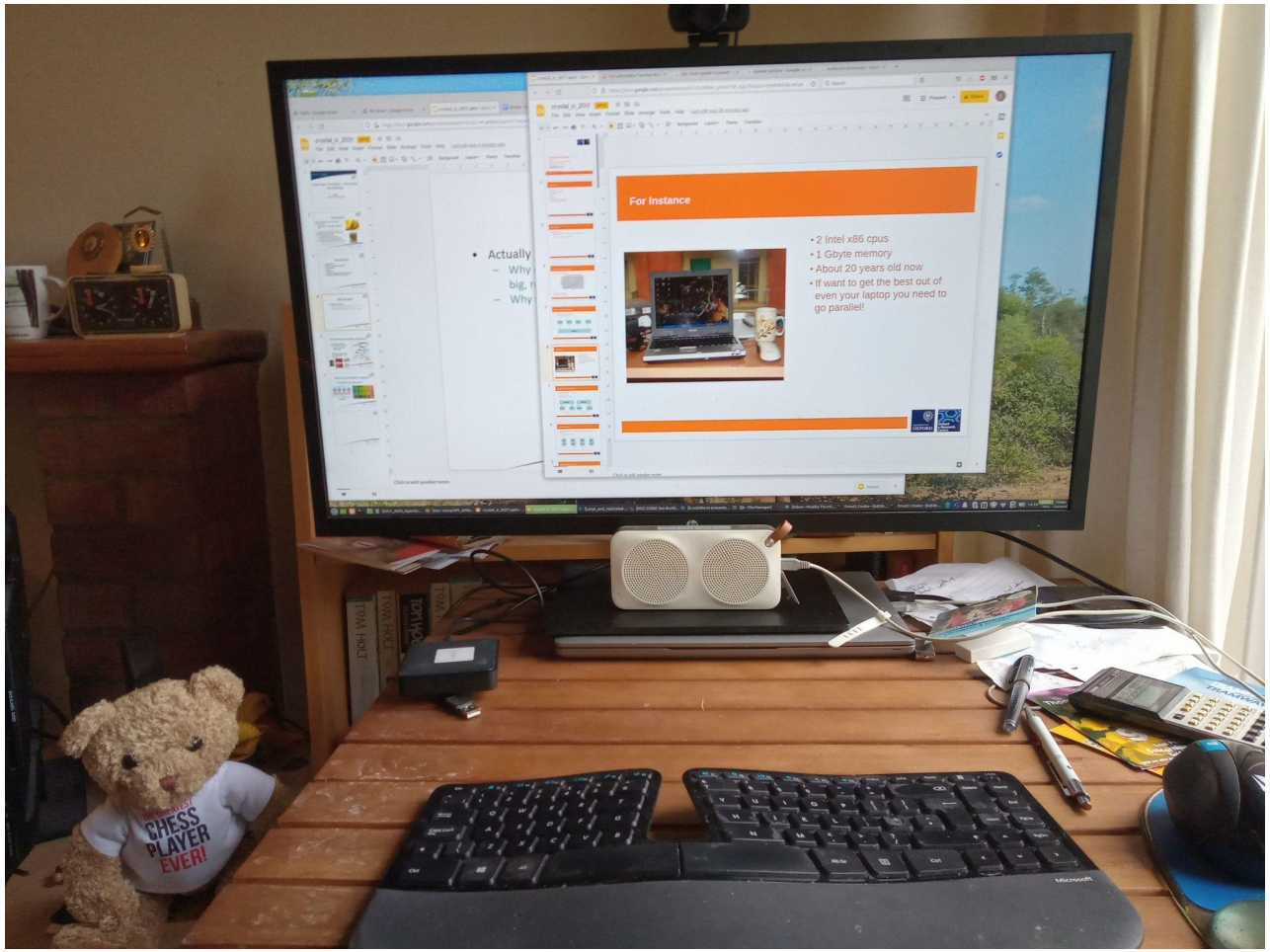
- The building block is the multicore processor
  - ~2-128 cores sharing the same memory





Science and Technology Facilities Council

For Instance

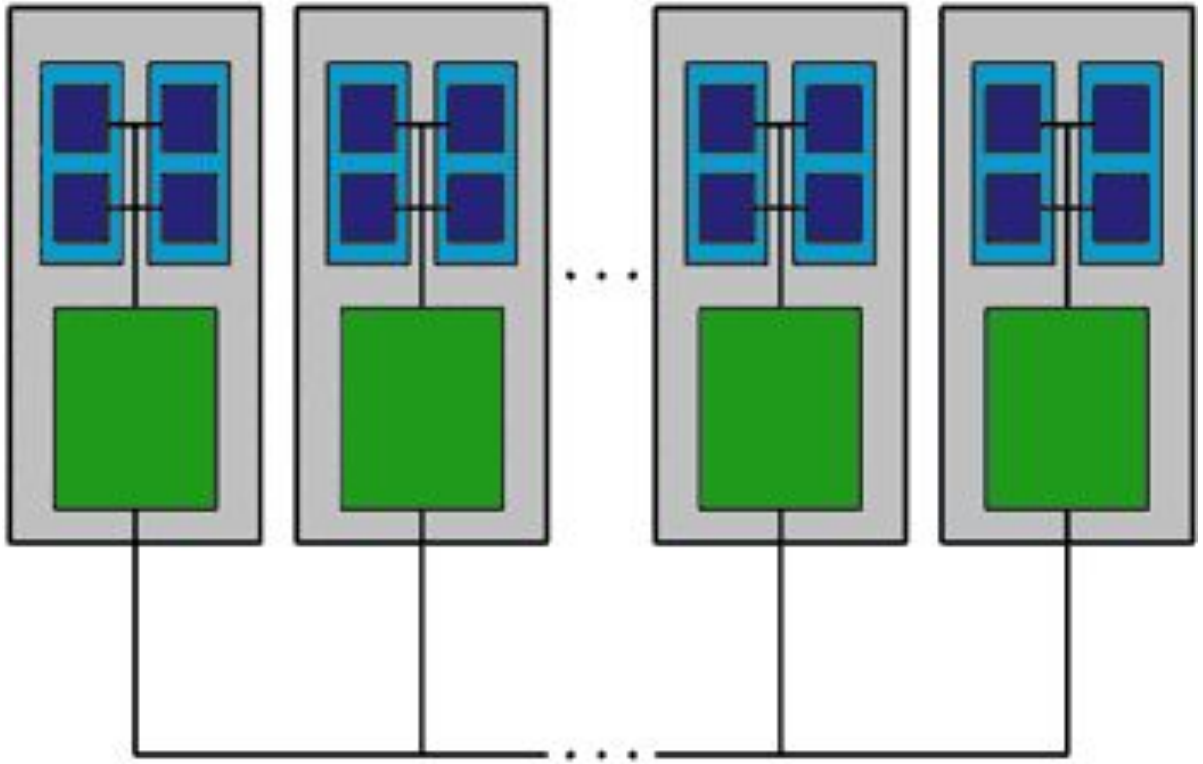


- 4 Intel x86 cpus
- 16 Gbyte memory
- About 3½ years old now
- If want to get the best out of even your laptop you need to go parallel!

Science and Technology Facilities Council

What Is In A Parallel Computer?

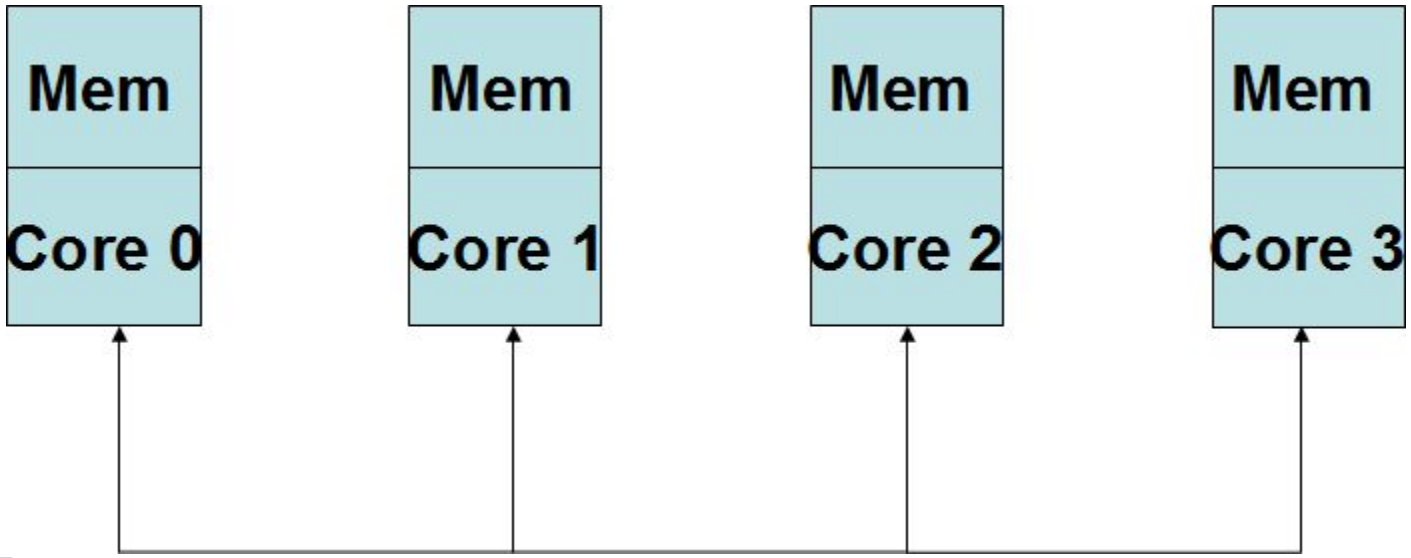
- These are clustered together with an interconnect
  - E.g. ethernet, infiniband
- A good one is expensive!
- Nodes MUST use the interconnect to “talk”



Science and Technology Facilities Council

How We Shall View It Initially

- Initially We shall take the simplified view of it being a number of cores joined together by the interconnect
  - I.e. ignore multicore
  - c.f. The internet



Why use Parallel Computers?

- Faster time to solution
  - Jobs that takes days or weeks take hours
- More Memory so larger jobs
  - 32 nodes means 32 times more memory so one hopes one can run a larger job
- BUT more complex to run
  - Depends on local set up
- And more difficult to get the best out of the program
  - Hope to give some tips on how to do this

Always Parallel?

© Scott Adams, Inc./Dist. by UFS, Inc.

Science and Technology Facilities Council

When Parallel?

- Do you always want to run in parallel?
- Very rare today when you don’t want to
- 10 people do not necessarily get a job done 10 times quicker
  - You have to have enough work to keep 10 people busy!
- Similarly 10 cores does not mean the computation will run 10 times quicker
  - You have to have enough work to keep 10 cores busy!

When Parallel

- Corollary: Do you always want to use all the cores available?
- NO!
- There maybe enough work for 10 cores but not 100
  - Overhead (e.g. use of the interconnect) may start to dominate
- Using more cores might make your program SLOW DOWN
  - Work often ~O(N/P) overhead often ~O(P)

Types Of Overhead

- Won't go into detail but reasons for overhead in parallel programs include
  - Amdahl's law – not all the program is parallelised
  - Load balance – not all tasks are equal
  - The interconnect – necessary evil but slow …
  - Memory bandwidth contention
  - I/O
  - O/S jitter
  - Phase of the moon
  - The computer hates you
  - ...

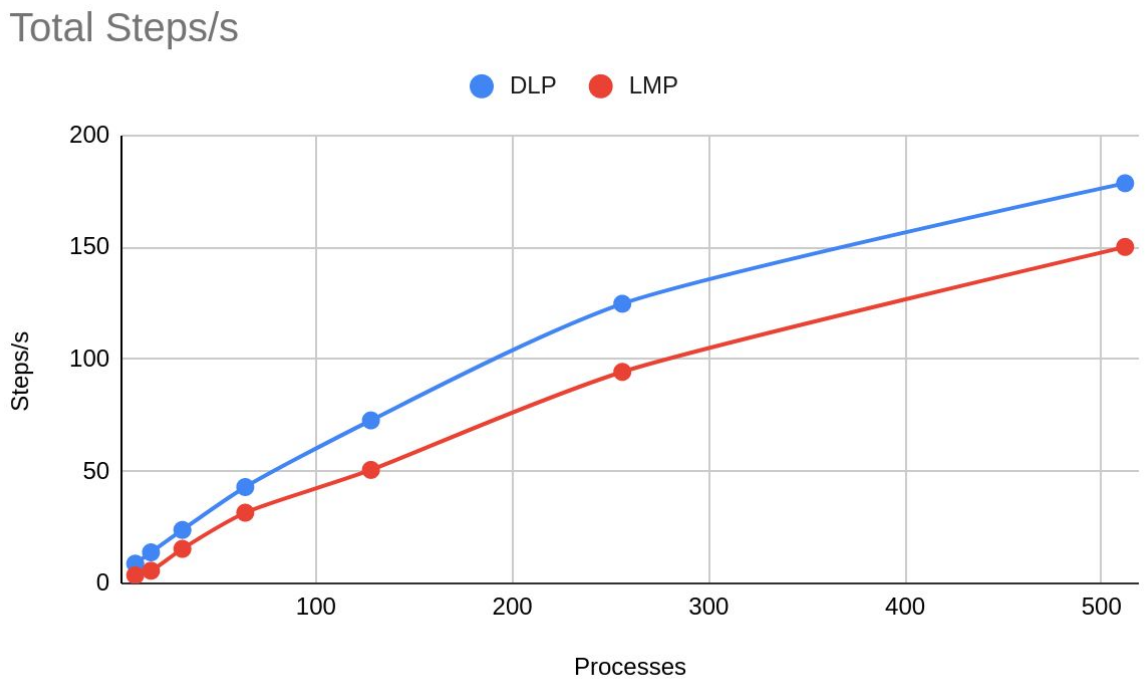
When Parallel

- Work ~O(N/P), overhead ~O(P)
- Corollary: As N increases the overhead becomes relatively less important
- Most important lesson:

Parallel Computing Is Best For Solving Large Systems

Trying To Quantify It

- So 100 cores doesn't mean 100 times faster
- And each job will have its own characteristics
  - So each job may be best run on a different number of cores
- But what do we mean by “best”?
- Need to measure something!
  - Speed up – applies to any code
  - CRYSTAL– SCF cycles per unit time, or geometries per unit time, may be of more interest



Science and Technology Facilities Council

Crystal – Parallel Implementations

- Pcrystal
  - Replicated data
  - All of CRYSTAL implemented
  - Good for small to medium sized problems on small to medium core counts
- MPPcrystal
  - Distributed data
  - Most of CRYSTAL implemented
  - Good for medium to large problems on medium to very large core counts
- New! OpenMP New!
  - CRYSTAL23
  - Will go into later

CRYSTAL – Basic Algorithm

- $H_R=P_R.I_R$
  - I_R sum of *independent* integrals
- At each k point *independently*
  - $H_K=FT(H_R)$
  - Solve $H_KQ_K=S_KQ_K\Lambda_K$
  - Form P_K from Q_K,Λ_K
- $P_R=FT(P_K)$
- Repeat until converged

Pcrystal

- Standard Compliant
  - Fortran 2003 + MPI 2
  - Should compile and work everywhere

- Replicated Data
  - Each processor has a complete copy of all objects used by the code
  - Make implementation simple
  - But limits size of system that can be studied

Pcrystal - Parallelism

- Generating the Kohn-Sham matrix
  - Each integral independent so can give a subset to each of the cores
  - DFT an integration over a grid – each point independent so each core is responsible for a different set of points
- Almost perfectly parallel!
  - Need to replicate at end
  - Limit on scaling is load balancing

Pcrystal - Parallelism

- Each K point is (almost) independent of the others
  - So each core can deal with a subset
  - Again need to replicate at the end
- But number of k points limits parallelism
- And large systems require few k points
  - And in the limit only 1
- Limit on size of system as each core holds a complete copy of the large matrices
  - However many cores used

Running Pcrystal

- How to run will depend on your local set up
  - Ask other users or look at web pages etc. for how to run
  - Will usually need a batch script if running on a cluster
  - The usual command to run Pcrystal will be something like
- mpirun -np 4 Pcrystal
  - BUT note the input file MUST be called INPUT
  - (and every core must be able to see it)

Pcrystal – Changes to your INPUT

- Strictly NO change is required to your INPUT file
- HOWEVER I very strongly recommend running in direct mode for parallel jobs to avoid I/O:
  - I think this is the default now

Pcrystal - Summary

- In general scales well provided integrals dominate
  - But k point limitation provides limit on scaling

- Limit on size of job is no bigger than that which serial CRYSTAL can cope with

MPPCrystal

- Again uses common standards
  - Fortran 2003
  - MPI 2
  - ScaLAPACK 1.7
    - http://www.netlib.org/scalapack/
- Distributed data
  - Each core only holds part of the large data structures
  - More cores, more memory, LARGER JOBS
  - More complex to implement

MPPCrystal - Parallelism

- Kohn-Sham Matrix Build – same as Pcrystal
- Linear Algebra
  - Where possible exploits k-point parallelism
  - But within each k-point exploit further levels of parallelism
    - More communication so more overhead
    - So need reasonable system size
    - $O(N^3)$ work, $O(N^2)$ communications
  - BUT not limited to just number of k-points
  - Great for large Γ point calculations!
- Very rough estimate – good while $N_S * N_K * N_{AO}/P \gtrsim 20$
  - But very machine and case dependent

MPPCrystal – other issues

- By default runs direct
  - Designed for 1000s cores – the disks would get very upset!
- Most but not all of CRYSTAL implemented
  - Will fail quickly and cleanly if the requested feature is not implemented.
- Running is just the same as PCRYSTAL
  - Need MPP binary
  - Remember input must be called INPUT

MPPCrystal - Summary

- Good for larger systems and large core counts, but overhead in linear algebra may dominate for small to medium sized systems
  - Especially for high symmetry systems

- Distributed data approach means that given enough cores can solve very large system sizes – current record is a few SCF cycles for over 500,000 basis functions

Properties

- Both Pcrystal and MPPcrystal write final output files that are in exactly the same form as the serial code
- So you run properties just as before
- But might take a long time and take a lot of memory
- So Pproperties - replicated data, like Pcrystal
- And MPPproperties
  - Distributed data like MPPCrystal
  - Supports NEWK, BAND, DOS, ECH3, POT3

How To Get The Best Out Of Them

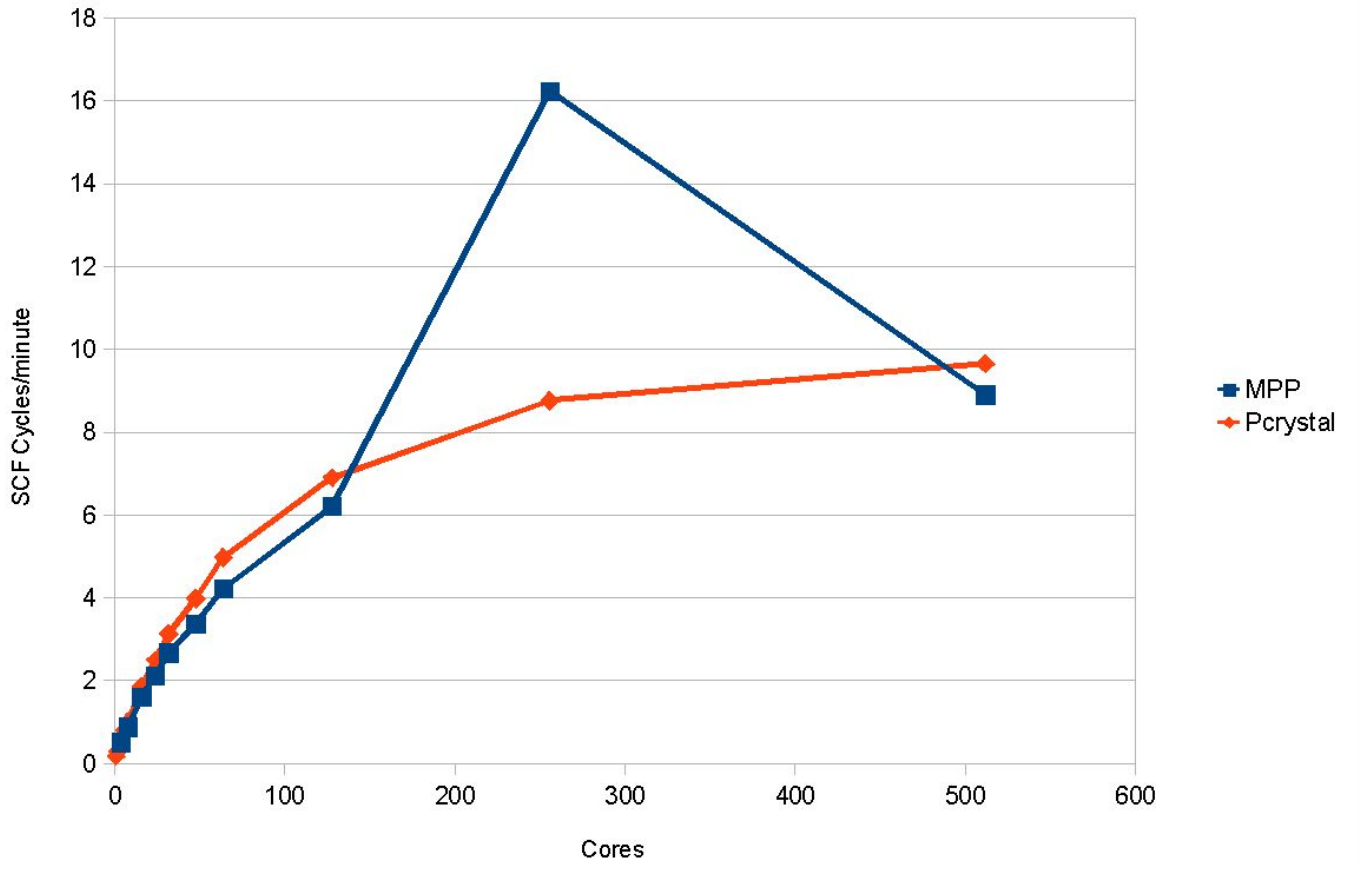
- I'll use a simple case study on how to get the best out of the codes, and so choose which one to use and how many cores
- Basic procedure is
  - A series of short runs (e.g. 1 geometry, or small number of SCF cycles) on differing number of cores to look at the scaling
  - Look at the output to see if we can work around any warnings
  - New set of short runs with improved input
  - From that data select PCRYSTAL or MPPCrystal and the number of cores to be used

An Example

- I'll use a “real” test case I got from a user
  - Organic crystal used in a drug study
  - 624 basis functions, 8 irreducible k-points
  - Real calculation a geometry optimisation
  - For performance tests just run the first SCF
  - 16 cycles

- A bit old now but the principles haven’t changed

First Performance Figures



Science and Technology Facilities Council

What Can we improve?

- So far Pcrystal looks better
  - System too small for diagonalisation in MPPcrystal to work well
- EXCEPT at 256 cores. What's going on?
- At 128 cores we see in the output

```
WARNING **** MPP **** ALL K POINTS DONE BY ALL
PROCESSORS
```

- At 256 we see

```
INFORMATION **** MPP **** MPP running in k
point/spin parallel mode
```

- Extra performance due to code deciding the number of cores good for k-point parallelism

The Art of Gentle Persuasion

- How to force it to go k-point parallelism?
- First in the last section put

```
CMPLXFAC
2
```

- This means you estimate your complex k points are twice as expensive as purely real ones
- Then look at the nature of your k-points

```
*** K POINTS COORDINATES (OBLIQUE COORDINATES IN UNITS OF IS =  3)
  1-R(  0  0  0)   2-C(  1  0  0)   3-C(  0  1  0)   4-C(  1  1  0)
  5-C(  0  0  1)   6-C(  1  0  1)   7-C(  0  1  1)   8-C(  1  1  1)
```

Science and Technology Facilities Council

Costing

```
*** K POINTS COORDINATES (OBLIQUE COORDINATES IN UNITS OF IS =  3)
  1-R(  0  0  0)   2-C(  1  0  0)   3-C(  0  1  0)   4-C(  1  1  0)
  5-C(  0  0  1)   6-C(  1  0  1)   7-C(  0  1  1)   8-C(  1  1  1)
```

- So we have 1 real k point and 7 complex
- So the unit cost is 2*7 + 1 = 15
- So run on multiples of 15 cores and the code should decide to go k point parallel
- Why not automatic?
  - Partially daft default I need to fix
  - Partially 'cos don't know number and nature of the k points until after chosen number of cores
  - Fix in the pipeline but not the next release

More Modern Version ...

Recent versions of the code print out a message similar to the below to help you (n.b. Not for the example discussed above):

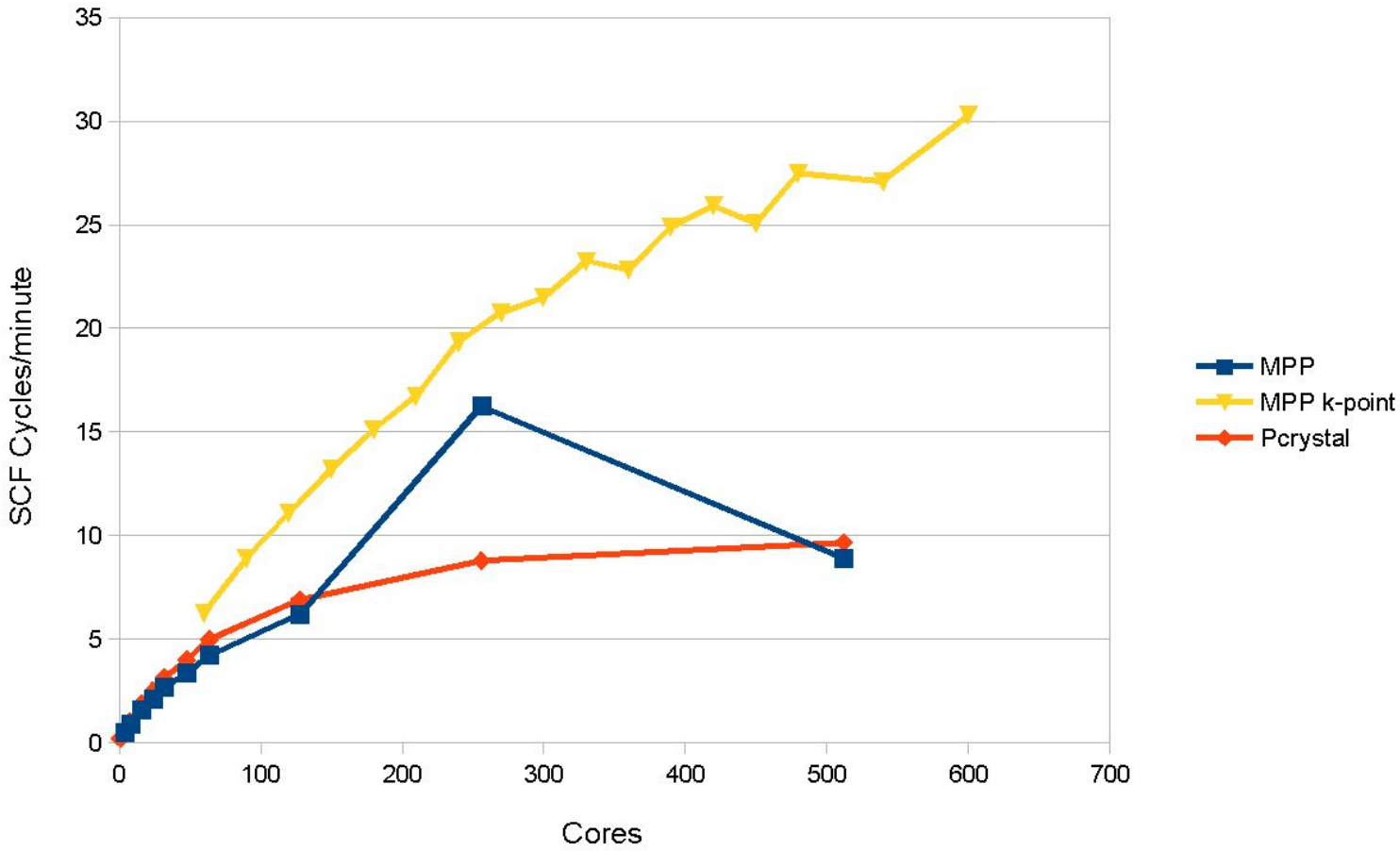
```
*****************************************************************************
  Please, consider that for this calculation a double level of parallelism
  can be exploited by using a number of processors given by:

     N_proc =    44 x a_non_prime_integer (as 4, or 6, or 8, or 9, ...)

  For instance, N_proc =     176, or     264, or     352, or     396, or ...
                N_proc =    2816, or    5632, or    11264, or   22528, or ...
                N_proc =    45056, or   90112, or ...
*****************************************************************************
```

Science and Technology Facilities Council

New Performance



Science and Technology Facilities Council

So …

- With a bit of work we have increased our number of SCF cycles from 10 per minute to around 30!
  - And the best code is now MPPcrystal
- Of course scaling is not perfect – up to you to make decision where is the best for you, especially if paying for the time
- Rough estimate above gives 250 cores for MPP
  - Not too bad
- Also note in this case the user didn't “go mad” with the shrinking factor …
  - Diagonalisation can be expensive!

MPP Examples and Latest Developments

- Crambin
  - A small protein
- MCM-41
  - Mesoporous aluminosilicate
- ***New*** OpenMP (Barry Searle)

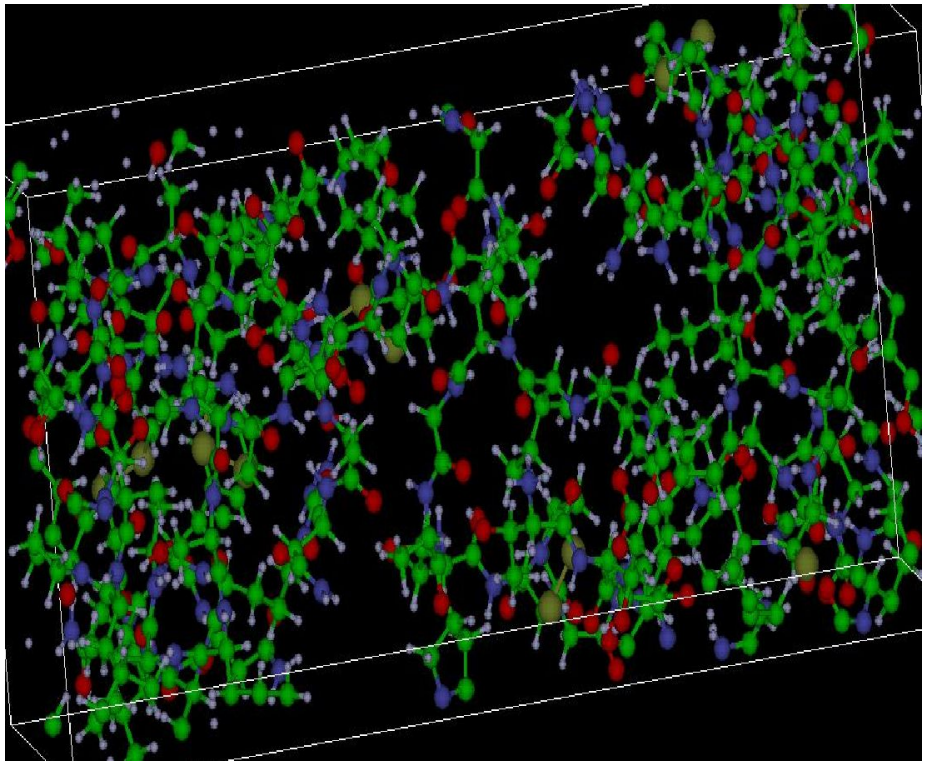
Crambin

- Small protein (46 residues)
- Crystal structure determined to very high resolution

Science and Technology Facilities Council

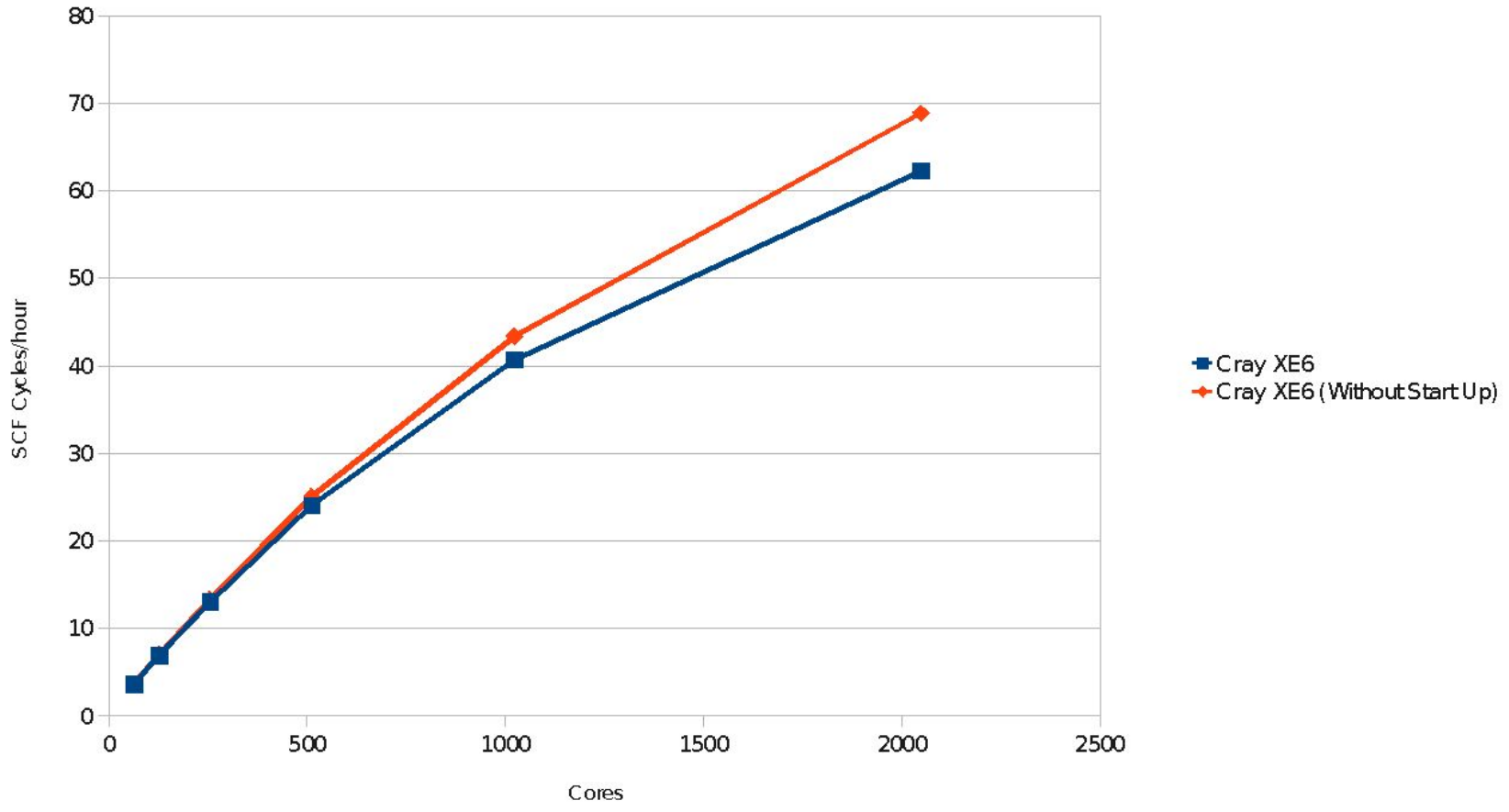
Crambin

- 1284 atoms
- 6-31G** basis set
- 12354 functions
- All calculations B3LYP
- Latest calculations include a full structural optimisation including water of crystallisation
  - Piero Uliengo *et al.*



Science and Technology Facilities Council

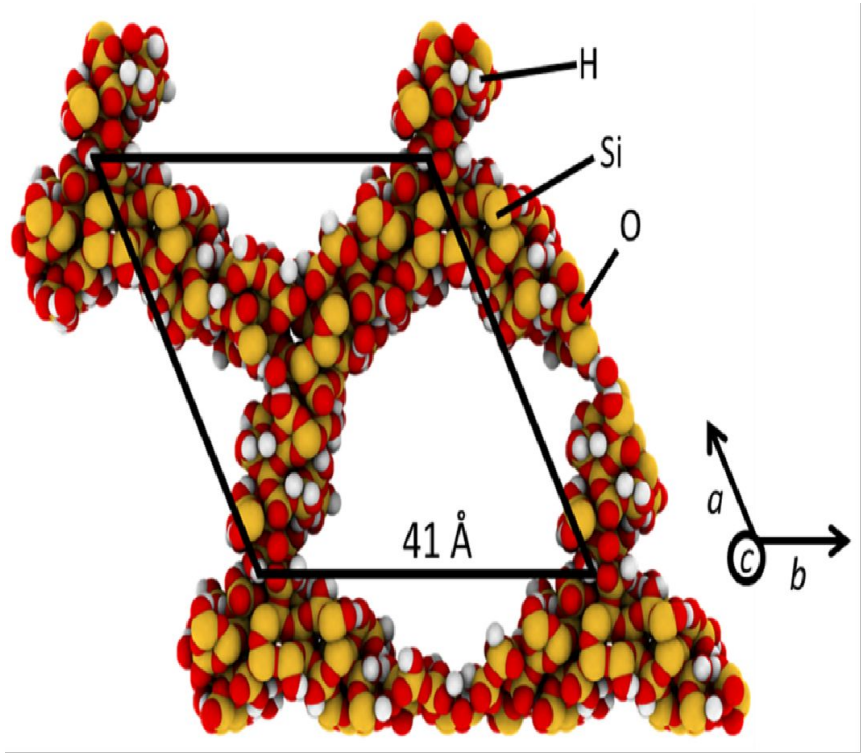
SCF Scaling



Science and Technology Facilities Council

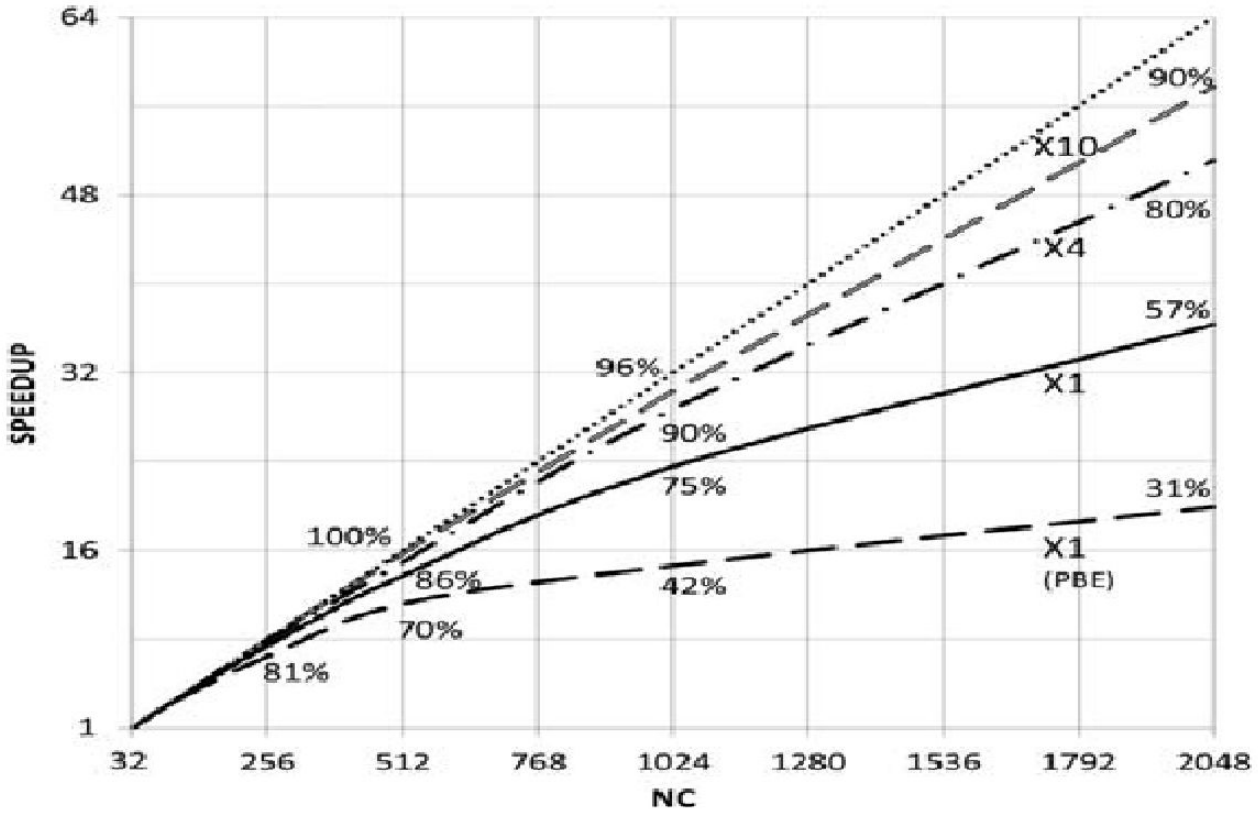
MCM-41

- Amorphous at short range
- Long range order of pores
- Model developed by Piero Ugliengo at et al. At the University of Turin
- 579 atoms, 7790 basis functions
- Also looked at supercells



Science and Technology Facilities Council

Scaling as a Function of System Size

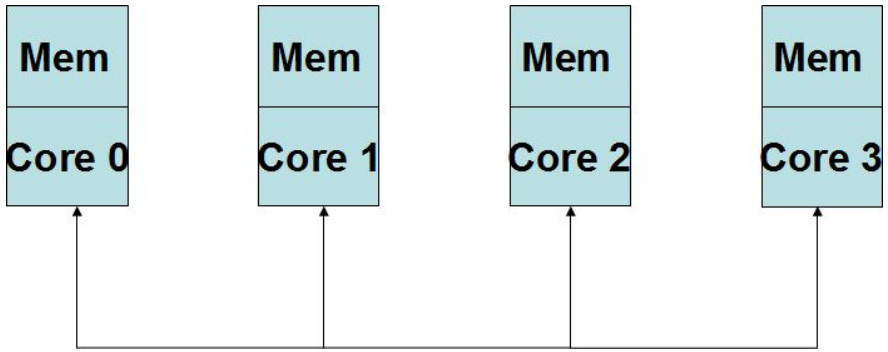


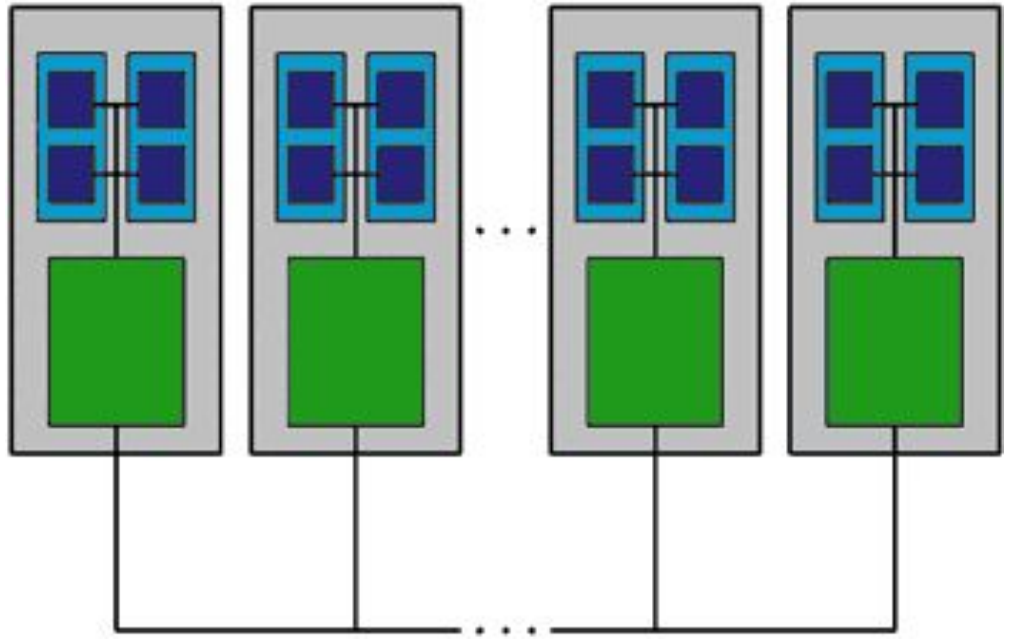
Science and Technology Facilities Council

Introduction of OpenMP

- Limit of the size of system that can be studied by MPPCrystal is due to the memory used by the integrals
  - Same as PCRYSTAL so replicated data
- In the next release this will be augmented by the availability of threads implemented by OpenMP (Barry Searle)
- Effectively instead of each core having a copy of all the data for the integrals, the data is shared by a number of cores, thus reducing the total memory

OpenMP



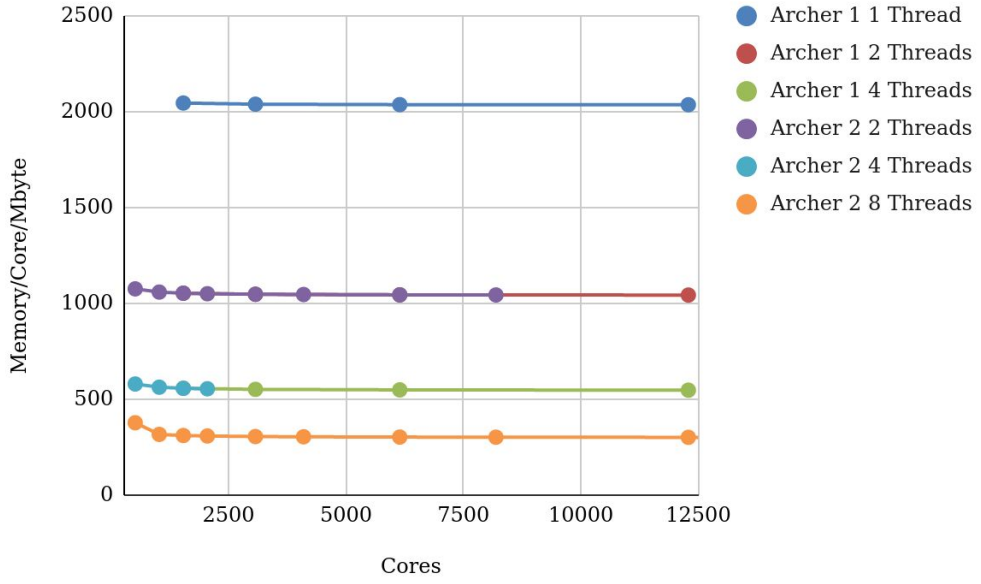


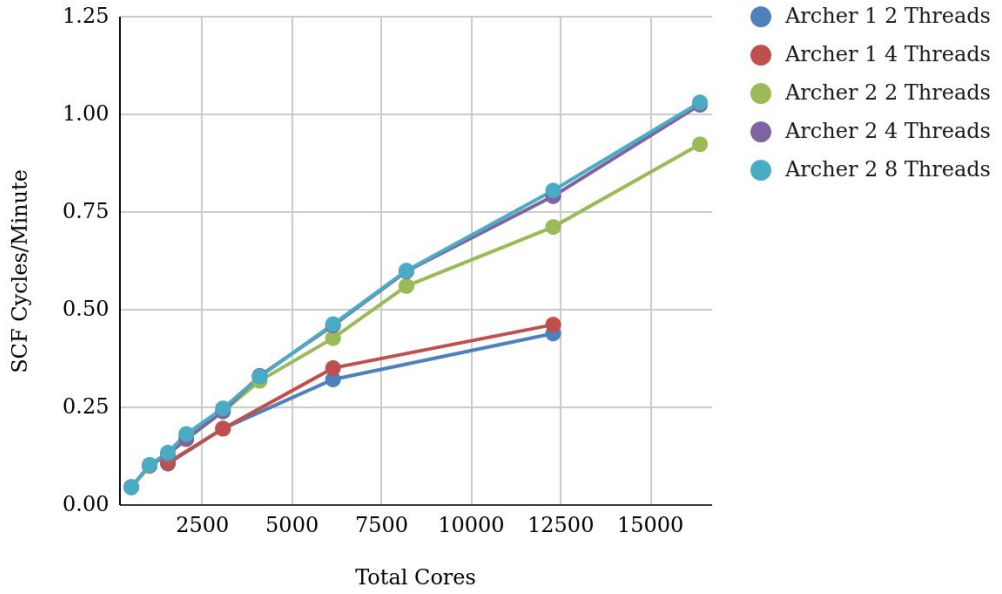
- Each process holds a whole copy of the replicated objects
- But ¼ of the number of processes
  - Work split across the group of 4 cores by threads
- So ¼ of the memory

Science and Technology Facilities Council

F Centre in LiF

- 6x6x6 Supercell with defect at centre, symmetry removed
- 23.94Å side of cell
- 1727 Atoms
- 21586 Basis functions, B3LYP Calculation

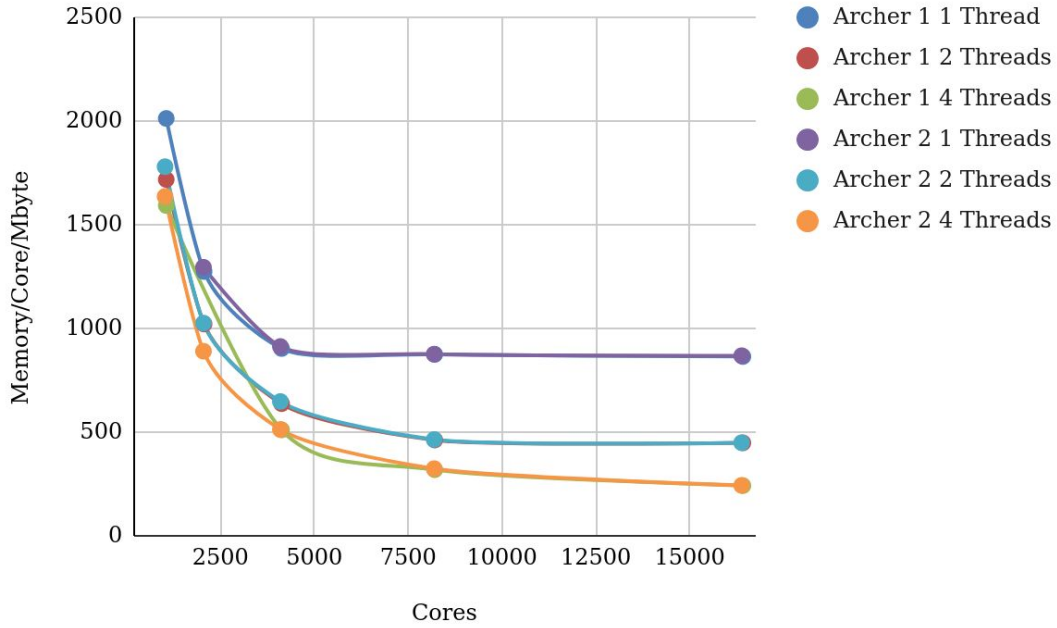


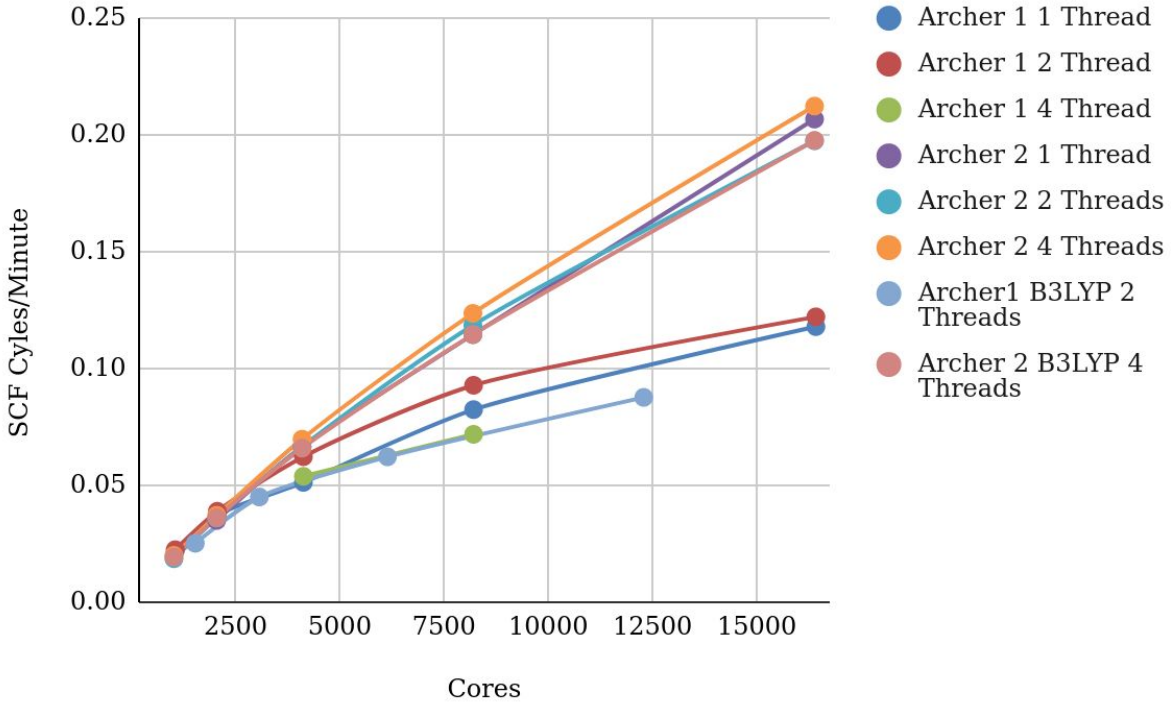


Science and Technology Facilities Council

MCM41x16 - PBE

- 9264 Atoms
- 124640 Basis Functions





Science and Technology Facilities Council

Summary

- Crystal can use 2 parallelisation strategies
  - Pcrystal
    - Good for medium size systems
    - Scalability limited by k-points
    - Can't study systems bigger than serial CRYSTAL
  - MPPCrystal
    - Good for large systems
    - Can scale very well for very big problems
  - OpenMP can be used with both
    - Reduces memory required
    - Can slightly improve performance
- In both cases a little bit of work on the input can markedly improve the performance